```
void hi( ) {
        syso ( "this. fuelCapacity);

Static void bye( ) {
        syso (this.fuelCapacity);
        }
```

error

we

* [we can't use this(), super(); in static methods because loaded with class, at that time objects were not created] loading time

Class: 19

13/05/2025

POLYMORPHISM

* Poly means many.
* Morphism means Forms. [is a Greek word]
* Polymorphism — Many forms.
* Many solutions for same problem — Polymorphism.
* It allows us to write same method in multiple forms.
* Types: 2

(* we can overload final method)

[* we can overload a constructor in Java]

1. Compiletime / static / Early Binding polymorphism — Method Overloading
2. Runtime / Dynamic / Late Binding polymorphism — Method Overriding.

1. Method Overloading: (Always methodname should be same).

* One method can have multiple forms, with different parameters (change the count of parameters)

```
int sum( int a, int b) {
    }
int sum( int a, int b, int c) {
    }
....
```

⇒ parameterised methods. like parameterised constructor (Constructor overloading).

*b) We can write same count of parameterised method with changing datatypes.

→ Changing the datatypes of parameters.

Ex:
```
void sum(int a, int b) {
}
void sum(string a, int b) {
}
```

*c) Changing the order of parameters.

Ex: ⇒
```
void sum(string a, int b) {
}
void sum(int b, string a) {
}
```

[* We can change order of parameters only if those are of different datatypes.]

⇒
```
void sum(int a, int b) {
}
void sum(int b, int a) {
}
```

* This will not allowed due to same datatypes.
* there is no difference in calling those methods as sum(4,5) & sum(5,4) - due to this JVM enter into ambiguity stage then gives error.

* The behaviour of parameterised method can be known during compile time by JVM - so it is called as compiletime polymorphism. (It (JVM) knows prior) & Early Binding polymorphism. (it knows before execution).

*d) We can change returntype & use the same method. but doesn't decide the polymorphism.

```
void sum(int a, int b) {
}
int sum(int b, int a) {
}
```

Ex: package com;

public class Calculate {

static int ~~void~~ sum ( ) {

int a = 10+20;

return a;

}

Psvm {

int sum = Calculate.sum ( ) ;

syso(sum); → output: 30

}

}

[* Polymorphism decicled only with parameters]

Ex: package com;

public class Calculate {

static int sum ( ) {

int a = 10+20;

return a;

}

static int sum(int a, int b) {

int c = a+b;

return c;

}

static void sum (int a, int b, int c) {

int d = a+b+c;

syso(d);

}

static void sum ( ) {

(error)

}

```
public static void main (String [ ] args) {

int sum = Calculator. sum ( )
                                    Output: 30
    syso ( sum) ;                           15
                                            60
  sum = Calculator. sum ( 10, 5);
    syso ( sum);
  Calculator. sum (10, 20, 30);
```

Ex: Environment (Version Control System – Git | Git Hub – which manages our code & stores in a cloud of Git Hub) – to share code with different developers to make changes & updates (to roll back).

(Accessible to End users)
- → Local (our system)
- → Dev (Developers) access
- → QA (Testing department)
- → pre prod
- → UAT (Users Acceptance Test - client)
- ← prod stage (new updates)

⇒ Git – Local (in our system only) – like eclipse code
↓ push to
Git Hub – Coloud (can share).

Ex:
```
// local
void getCassandraDbConnection(Username, password) {  → Local

}
// Developer
void getCassandraDbConnection (Username, password, hostname) {  → Dev

}
// prod
void getCassandraDbConnection(Username, password, hostname, ssl) {  → prod
    https://                                  ↓ (send a certificate to server/client)
}
```

* Overload happens in same class.
* We can overload a main method in Java, our JVM will not get confused, it will search only for psvm( ). which accepting (String[ ] args).
* JVM consider only main method, if we overload the main method.

Method Overriding: Class:20 14/05/2025

* Changing the implementation from parent to child class.
* It works only in Inheritance (Parent & Child class).
* This comes into picture only when there is inheritance.
* We can override a parent class method in child class by giving your own implementation.

[* We cannot override final methods also]

Ex:

```
package com;
public class Remote {                    [* We cannot override constructor]
    void turnOn() {
        syso("Turned on....");
    static void turnOff() {
        syso("Turned Off ---");
    }
}
```

```
Package com;
public class TVRemote extends Remote {
    @Override
    void turnOn() {
        super.turnOn();
        syso("TV Turned On ---")
    }
    static void turnoff() {
        syso("Tv Turned off ---");
    }
}
```

annotations

used to call parent class methods.

Method Hiding.

```
package com;
public class AcRemote extends Remote {
@Override
    void turnOn() {
        syso("AC Turned on..")
    }
}
```

a) Always methodname should be same.
b) We cannot change returntype.
c) We cannot change parameters, given in method.
d) We can Change only implemention inside the method.
e) We can change ~~use~~ accessmodifiers (public, private....)
f) It goes with instance methods only.
g) We need inheritance for Overriding.

* @Override is not mandatory to write in program, but for good approach we have to use it.

* It mentions above method only.

```
package com;
public class Test {
    psvm {
        Remote remote = new Remote();
        remote.turnOn();
    }
}
```

→ output: Turned on....

```
TvRemote tv = new TvRemote();
tv.turnOn();
```

→ output: Tv Turned on....

↳ Output: Turned on....
Tv Turned on....

* Behaviour of method is decided in Runtime by JVM, so it is called as Runtime polymorphism.

* The Binding happens lately after creation of an object so, it is called as Late Binding.

* We can't be able to Override static methods in Java due to static methods get memory at compile time but overriding will happen at runtime. — Method Hiding

```
tv.turnOff();
```

→ output: Tv Turned off.

Method Hiding:

* When we write a static method same as method in parent class, it does not indicate overriding, it indicates that we are hiding the static methods in parent class.

→ It works only with static methods.

```
public static void main(String[] args) {   → JVM call this
    main(10);
}
public static void main (int[] args) {
}
public static void main (int args) {
    syso(args);
}
```

we need to call manually.

* we can overload main method, but JVM consider only main method.

* we cannot override main method, due to, static methods cannot be overrided. (main method is a static method)

→ static methods only be method Hiding. ↓ because it hides main method.

* **Final Keyword:**

* final, finally, finalise — 3 keywords.
  - finally → exception Handling
  - finalise → Garbage collection.

final: Variables

→ If we make the value constant, we have to use final keyword.

* When we declare final keyword, we cannot change values.
* final variables can be declared when we assigned the variable.
* We cannot declare final variables without initialising.

Ex: package com;
public class Final {

int a = 10;
final int b = 15;
final String password = "*****";

public static void main(String[] args) {
Final obj = new Final( );
Syso(obj.a); Output: 10
obj.a = 20; 20
Syso(obj.a); 15
Syso(obj.b); *****
obj.b = 20 (we cannot change the value of
error final variable).
Syso(obj.password);

Methods:

* final keyword can be used with methods also.

Ex: ✓ final void hi( ) {
Syso("Hii....");

* we cannot override final method.

Ex. package com;
public class child extends Final {
✗ void hi( ) { (we cannot override final method
error in Java)
Syso("Hi...");

* We can overload final method by changing parameters.

✓ void hi(String name) {
Syso(name)
}.

* Class [final keyword in class].

we will not be able to inherit the class when we used final keyword in class.

Ex: package com;
public final class Final {

}

package com;
public class child extends Final {
error (we cannot override when we use final keyword in class).

}

15/05/2025

Class: 21

Access Modifiers – 4

* These decider the access of your variables, methods...
* They are – 4.
  1. public
  2. Private
  3. Protected
  4. Default.

1. Public:

* we can use public with, Variables, methods, class, constructor.
* Public is a keyword.
* When we use public beside variable, method, etc. it will be accessible across our project.
*

Ex:

```
package com;
public class Class1 {
    public int a = 10;
    public void hi() {
        syso("Hi....");
    }
    private int b = 20;
    Private void bye() {
        syso("Bye...")
    }
}
```

→ Constructors.

```
private Class1() {
    syso(
}
protected int c = 30;
protected class1() {
}
int d = 40; (default variable)
}
```

```
public static void main(String[] args) {
    Class1 obj = new Class1
    syso(obj.a);        → output: 10
    obj.hi();                Hi....
    obj syso(obj.b)          20
    obj.bye();               Bye...
    syso(obj.c);             30
    syso(obj.d);             40
}
```

```
package com;
public class Class2 {

}
public static void main(String[]args) {
    Class1 obj = new Class1();
    syso("From class2:" + obj.a);
    obj.hi();
    syso(obj.b);
    obj.bye();
    obj.syso(obj.c);
    syso(obj.d);
}
```

[If the constructor is private, we can't be able to access in different class]

output:
From class2: 10
Hi....
30, 40

[even if we in same package, we cannot access variable, in different class]. methods.

package com.flm; → import com.class1; [∵ class1 is coming from different package].
public class Class3 {

}

public static void main (String[] args) {
Class1 obj = new Class1();
Syso("from class 3: " + obj.a);
Obj.hi(); (we access method due to it is a public method)
obj.b;
obj.bye(); (error)
Syso(obj.c); → (∵ package is different)
Syso(obj.d); (error) → (due to different package).
}

output:
From Class 3: 10
Hi---

2) Private:

* It can be used with Variables, methods, constructor, but we cannot use private with class.
* We can be able to access only in the same class, when we mention private.
* JVM will not be able to load private class, with in the package, so, JVM will not allow to write private class.
* If the constructor is private in one class, we can't create an object of that class in another class.

3) Protected:

[(Single ton class) → We can create only one object access the project]

* It can be used with Variables, method & constructors.
* We can be able to access, with in the package.
* When we access in different package, if it is a child class.

public class 3 extends class1{ } [∵ It will access due to it is a child class coming from parent class]

psvm {
Class3 obj1 = new Class3();
Syso(obj1.c); → output: 30
obj1.class1();
}
}

4. Default:

* It can be used with Variables, methods, Constructors.
* We can access only with in the same package.
* Default can be used if there is no subclass.
* We can write class as default.
* We can't use access modifiers with Local Variables. (due to, local variable can be accessed with in the main method)
* We can't use access modifiers inside methods.

Encapsulation

* It is nothing but Hiding something (Variables/methods) for increasing security.
* We use encapsulation by making Variables private only.

Ex:

```
package com;
public class User {                    [:: Here User class is child of
// int age ;                                object class, since for every class
// String name;                             object class is a parent class]
   private int age;
   private String name;
```

getters & setters methods: to access those private Variables

```
public void setAge (int age) {
    if (age>0) {
        this.age = age;
    } → else {
        syso("wrong age passed");
    }
}
public int getAge ( ) {
    return this.age;
}
public void setName ( String name) {
    this.name = name;
}
public String getName ( ) {
    return this.name;
}
```

```
package com;
public class Tests {
    public static void main(String[] args) {
        User user = new User();
        user.age = 20;
        user.name = "Gayatri";
        // Syso(User.age);
        // Syso(user.name);
    }
}
```

(error)

output: 20 Gayatri

```
User.setAge(10); →
Syso(user.getAge);
```

output: 10

```
User.setAge(-10);
Syso(user.getAge);
```

output: Wrong age passed 0 (default value)

```
User.setName("Gayatri");
Syso(user.getName);
```

output: Gayatri.

* Encapsulation says only make your variable private & write setters and getters to not give direct access to other classes.
* Object class is the parent class of all classes in Java. (ctrl + shift + t) [JVM will be extended it].
* Object class is different from Object.
* Getters and Setters are written only for variables.
* Encapsulation is a process of wrapping of data & methods in a single unit. is ~~called~~